

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

C03: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks: 50

CASE STUDY:

Question 1

Case Analysis

A multinational software company has developed a large ERP application with **120 dynamically linked libraries**. Users experience **high startup time** while memory utilization remains low. The task is to determine whether **static linking or dynamic linking** is better and recommend an optimized loading strategy.

Static Linking vs Dynamic Linking

Static Linking

Libraries are copied into the executable at compile time.

Larger executable size.

More memory usage because every program has its own copy.

Faster execution after loading.

Updating a library requires recompilation.

Dynamic Linking

Libraries are loaded during execution.

Smaller executable size.

Less memory usage since libraries are shared.

Slight delay while loading libraries.

Library updates do not require recompilation.

Optimized Loading Strategy

- Use **dynamic linking** for shared libraries.
- Load only required libraries during execution (lazy loading).
- Keep frequently used libraries in memory.
- Share common libraries among multiple processes.

Memory Diagram

Static Linking

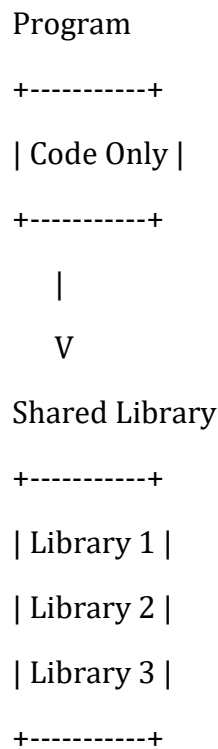
Program

+-----+

| Code + All Libraries |

+-----+

Dynamic Linking



Recommendation

Dynamic linking is the better choice because it reduces memory usage, allows library sharing, and simplifies software updates. Startup time can be improved using lazy loading and preloading frequently used libraries.

Question 2

Case Analysis

A cloud server with **128 GB RAM** executes AI, database, and web applications simultaneously. Despite having large memory, allocation failures occur due to fragmentation. The objective is to design a memory management strategy.

Causes

- Dynamic memory allocation creates fragmented free spaces.
- External fragmentation occurs in contiguous allocation.
- Internal fragmentation occurs with fixed-size memory blocks.
- Heavy multitasking increases memory requests.

Fragmentation

External Fragmentation

|P1|Free|P2|Free|P3|Free|

Total free memory is sufficient
but not contiguous.

Internal Fragmentation

Memory Block = 8 KB

Process uses = 6 KB

Unused = 2 KB

Memory Management Strategy

- Implement paging to remove external fragmentation.
- Use dynamic memory allocation.
- Merge adjacent free blocks.
- Apply demand paging.
- Monitor memory utilization continuously.

Recommendation

Paging with dynamic storage management minimizes memory wastage, improves throughput, and supports multiple large applications efficiently.

Question 3

Case Analysis

An airline reservation system experiences delays because the operating system frequently swaps processes between RAM and disk during heavy booking periods.

Effects of Swapping

- CPU waits while data moves between RAM and disk.
- CPU utilization decreases.

- Response time increases.
- System throughput decreases.

Process Flow

RAM

↓

Swap Out

↓

Disk

↓

Swap In

↓

RAM

Methods to Reduce Swapping

- Increase process priority for active tasks.
- Use demand paging.
- Apply Working Set Model.
- Reduce unnecessary background processes.
- Use SSD instead of HDD for swapping.

Recommendation

Demand paging combined with Working Set management reduces swapping overhead and improves CPU utilization without upgrading hardware.

Question 4

Case Analysis

A hospital management system uses contiguous memory allocation. Although enough free memory exists, large applications cannot be loaded. The issue is related to fragmentation. Compare allocation strategies and recommend the best one.

Reason

External fragmentation causes free memory to be scattered into small blocks.

Comparison

Method	Advantages	Disadvantages
First Fit	Fast allocation	Creates fragmentation
Best Fit	Less wasted space	Produces many small holes
Worst Fit	Leaves large holes	Poor memory utilization
Next Fit	Faster than First Fit	Higher fragmentation

Example

Free Memory

20 KB | 10 KB | 30 KB | 15 KB

Need = 25 KB

First Fit → 30 KB

Best Fit → 30 KB

Worst Fit → 30 KB

Next Fit → Depends on pointer

Recommendation

First Fit is the preferred technique because it offers fast allocation with acceptable memory utilization. For long-term performance, paging can further eliminate external fragmentation.

Question 5

Case Analysis

A banking application migrates from contiguous memory allocation to paging. Compare both approaches under heavy transaction loads.

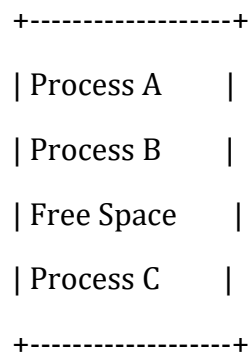
Comparison

Feature	Contiguous Allocation	Paging
External Fragmentation	High	None

Feature	Contiguous Allocation	Paging
Internal Fragmentation	Low	Possible
Memory Utilization	Moderate	High
Page Table	Not required	Required
Performance	Slower for large applications	Better multitasking
Memory Allocation	Difficult	Flexible

Memory Diagram

Contiguous Allocation



Paging

Pages

P0 → Frame 3

P1 → Frame 7

P2 → Frame 1

P3 → Frame 5

Recommendation

Paging is the better memory management technique for banking applications because it eliminates external fragmentation, improves memory utilization, supports heavy transaction processing, and enables efficient multitasking despite the small page table overhead.

Question 6

Case Analysis

A gaming company develops an open-world game with independent modules such as **graphics, AI, physics engine, and audio processing**. The task is to analyze how **segmentation** improves memory organization compared to paging and explain logical-to-physical address translation.

Segmentation vs Paging

Segmentation

Divides memory into logical segments.

Segment sizes vary.

Supports logical organization.

Easier program protection.

Paging

Divides memory into fixed-size pages.

All pages are of equal size.

Focuses on efficient memory allocation.

Eliminates external fragmentation.

Segment Table

Segment Base Address Limit

Graphics	2000	500
AI	3000	400
Physics	4000	600
Audio	5000	300

Address Translation

Logical Address:

<Segment Number, Offset>

Example:

<2,150>

Segment 2 → Base = 4000

Physical Address = 4000 + 150 = 4150

Memory Diagram

Logical Memory

Graphics

AI

Physics

Audio

↓

Segment Table

↓

Physical Memory

Recommendation

Segmentation is suitable because each game module is stored independently, improving memory protection, modular programming, and easier debugging.

Question 7

Case Analysis

A software company uses **segmentation with paging**, but developers observe increased address translation time during debugging. Compare it with pure paging and pure segmentation, then recommend the best architecture.

Comparison

Feature	Paging	Segmentation	Segmentation with Paging
External Fragmentation	None	Present	None
Internal Fragmentation	Present	None	Small
Protection	Moderate	High	Very High
Address Translation	Fast	Moderate	Slow
Memory Utilization	High	Moderate	Very High

Advantages

- Better memory protection.
- Efficient memory utilization.
- Supports large applications.
- Logical program organization.

Disadvantages

- Two-level address translation.
- Increased hardware complexity.
- Slightly higher translation time.

Recommendation

Use **Segmentation with Paging** along with a **Translation Lookaside Buffer (TLB)** to reduce address translation time. This provides strong protection and efficient memory management.

Question 8

Case Analysis

A mobile operating system allows users to run many applications simultaneously, exceeding available physical memory. Explain how **virtual memory** enables multitasking and what happens if it is disabled.

Role of Virtual Memory

- Gives each application its own logical memory.
- Loads only required pages into RAM.
- Allows execution of programs larger than physical memory.
- Improves multitasking.

If Virtual Memory is Disabled

- Only programs that fit into RAM can execute.
- Fewer applications can run simultaneously.
- System performance decreases.
- Frequent "Out of Memory" errors occur.

Improvements

- Use demand paging.

- Apply efficient page replacement (LRU).
- Compress inactive memory pages.
- Use SSD for faster swapping.

Diagram

Application

↓

Virtual Memory

↓

RAM

↓

SSD (Swap Area)

Recommendation

Virtual memory is essential for mobile operating systems because it enables efficient multitasking and better memory utilization.

Question 9

Case Analysis

An online video editing application loads only required portions of large videos into memory. Users experience delays while editing. Analyze the impact of **demand paging** and suggest methods to reduce page faults without increasing RAM.

Demand Paging

- Loads pages only when required.
- Saves physical memory.
- Reduces startup time.

- May increase delays due to page faults.

Causes of Delay

- Frequent page faults.
- Large video files.
- Slow disk access.
- Inefficient page replacement.

Methods to Reduce Page Faults

- Use **LRU page replacement**.
- Prefetch nearby video frames.
- Increase page size where appropriate.
- Store swap files on SSD.
- Keep frequently used pages in memory.

Page Fault Process

CPU

↓

Page Not Found

↓

Page Fault

↓

Load from Disk

↓

Resume Execution

Recommendation

Demand paging should be combined with LRU replacement and SSD storage to reduce editing delays while maintaining efficient memory usage.

Question 10

Case Analysis

A data analytics server repeatedly loads and replaces the same pages while processing large datasets. Compare **FIFO, LRU, Optimal, and Clock** page replacement algorithms and recommend the most suitable one.

Comparison

Algorithm Advantages		Disadvantages
FIFO	Simple implementation	May remove frequently used pages
LRU	Keeps recently used pages	Higher implementation cost
Optimal	Minimum page faults	Impossible in real systems
Clock	Fast approximation of LRU	Slightly less accurate

Analysis

- FIFO may suffer from Belady's anomaly.
- LRU performs well when data is repeatedly accessed.
- Optimal is used only for comparison because future references are unknown.
- Clock provides good performance with lower overhead.

Recommendation

LRU (Least Recently Used) is the best choice because analytics workloads repeatedly access recently used pages. It minimizes page faults and improves processing speed. If implementation cost is a concern, the **Clock algorithm** is a practical alternative.

Question 11

Case Analysis

A university examination server crashes during online examinations because of excessive page faults. Investigate whether **thrashing** is responsible and design a memory allocation policy to eliminate it.

Thrashing

Thrashing occurs when the operating system spends more time swapping pages between RAM and disk than executing processes.

Symptoms

- Very high page fault frequency.
- Low CPU utilization.
- Slow response time.
- Continuous disk activity.

Analysis

Parameter	Effect During Thrashing
-----------	-------------------------

CPU Utilization	Decreases
-----------------	-----------

Page Fault Frequency	Increases
----------------------	-----------

Disk Activity	Very High
---------------	-----------

System Performance	Poor
--------------------	------

Memory Allocation Policy

- Allocate more frames to processes with high page faults.
- Use the **Working Set Model**.
- Monitor **Page Fault Frequency (PFF)**.
- Reduce the degree of multiprogramming.
- Suspend inactive processes temporarily.

Diagram

Process

↓

Page Fault

↓

Swap Page

↓

CPU Waits

↓

Thrashing

Recommendation

The server should use the **Working Set Model** with **Page Fault Frequency control** to reduce excessive page faults and prevent thrashing, thereby improving CPU utilization and system stability.

Question 12

Case Analysis

A cloud provider hosts hundreds of virtual machines (VMs) on one physical server. During peak hours, several VMs become unresponsive. Analyze the role of **working sets** and propose a memory allocation framework.

Working Set

The working set is the collection of pages actively used by a process during a specific period.

Role of Working Sets

- Keeps active pages in RAM.
- Prevents unnecessary page replacement.
- Reduces page faults.
- Prevents thrashing.

Dynamic Memory Allocation Framework

1. Monitor memory usage of each VM.
2. Calculate each VM's working set.
3. Allocate additional frames when required.
4. Reclaim unused memory from idle VMs.
5. Continuously monitor workload changes.

Framework Diagram

Virtual Machines

↓

Working Set Analysis

↓

Dynamic Frame Allocation

↓

Efficient Memory Utilization

Recommendation

A dynamic working-set-based allocation framework ensures each VM receives sufficient memory during peak workloads, improving responsiveness and preventing thrashing.

Question 13

Case Analysis

A multimedia company stores HD video files on HDDs and frequently accessed metadata on SSDs. Analyze how disk devices influence **virtual memory**, **swapping**, and **page replacement**. Recommend a suitable storage hierarchy.

Comparison of HDD and SSD

Feature	HDD	SSD
Access Speed	Slow	Fast
Swapping Speed	Low	High
Page Fault Handling	Slow	Fast
Reliability	Moderate	High
Cost	Low	Higher

Storage Hierarchy

CPU Cache

↓

RAM

↓

SSD

↓

HDD

Advantages

- Frequently accessed pages remain on SSD.
- Large video files are stored on HDD.
- Faster swapping improves virtual memory performance.
- Reduced page replacement delay.

Recommendation

Use SSD for swap space and frequently accessed metadata, while storing large multimedia files on HDD. This combination minimizes latency and improves overall memory performance.

Question 14

Case Analysis

A software company plans to redesign its operating system memory manager by integrating **dynamic linking, demand paging, segmentation, virtual memory, and SSD-based swapping**. Evaluate this integrated approach and identify bottlenecks.

Integrated Architecture

- Dynamic Linking → Shared libraries.
- Demand Paging → Load only required pages.
- Segmentation → Logical organization.
- Virtual Memory → Large logical address space.
- SSD Swapping → Faster page transfer.

Advantages

- Better memory utilization.
- Reduced startup time.
- Faster swapping.
- Improved multitasking.
- Better protection between program modules.

Possible Bottlenecks

- Large page table overhead.
- Increased address translation time.
- High SSD usage under heavy workloads.
- Additional management complexity.

Architectural Improvements

- Use a **Translation Lookaside Buffer (TLB)**.
- Apply the **LRU page replacement** algorithm.
- Use the **Working Set Model**.
- Optimize dynamic library loading.

Recommendation

An integrated memory management approach significantly improves performance and memory efficiency when combined with TLB optimization and intelligent page replacement.

Question 15

Case Analysis

An e-commerce platform experiences inconsistent response times during festival sales due to **dynamic library loading, page replacement, and swapping**. Develop a comprehensive optimization strategy.

Problems Identified

- Excessive dynamic library loading.
- Frequent page replacements.
- Heavy swapping.
- Increased response time.

Interaction of Components

Dynamic Linking

↓

Demand Paging

↓

Page Replacement

↓

Swapping

↓

Performance

Optimization Strategy

- Load frequently used shared libraries during startup.
- Use **Demand Paging** with **LRU page replacement**.

- Reduce swapping by using the **Working Set Model**.
- Store swap space on SSD.
- Continuously monitor page fault frequency.
- Increase memory allocation for high-priority services during peak traffic.

Benefits

- Faster application startup.
- Reduced page faults.
- Lower disk I/O.
- Better CPU utilization.
- Stable response time during heavy workloads.

Recommendation

A combination of **dynamic linking, demand paging, LRU page replacement, SSD-based swapping, and working-set-based memory allocation** provides the best performance while minimizing memory consumption during high-traffic periods.

Question 16

Case Analysis

A research laboratory develops a scientific simulation using very large matrices that exceed the available RAM. Compare **virtual memory, segmentation, paging, and segmentation with paging** and recommend the most suitable memory management model considering scalability, performance, and memory protection.

Comparison

Memory Management	Advantages	Disadvantages
Virtual Memory	Executes programs larger than RAM	More page faults if memory is low
Paging	Eliminates external fragmentation	Internal fragmentation and page table overhead
Segmentation	Logical organization and protection	External fragmentation

Memory Management	Advantages	Disadvantages
Segmentation with Paging	Combines protection and efficient memory use	More complex address translation

Analysis

- Scientific simulations require handling very large datasets.
- Virtual memory allows execution beyond physical RAM.
- Paging efficiently allocates memory without external fragmentation.
- Segmentation separates program modules logically.
- Segmentation with paging combines the benefits of both methods.

Memory Diagram

Scientific Simulation

↓

Virtual Memory

↓

Segmentation

↓

Paging

↓

Physical Memory

Recommendation

The laboratory should use **Segmentation with Paging** along with **Virtual Memory**. This provides scalability for large datasets, efficient memory utilization, and strong protection between different program modules.

Question 17

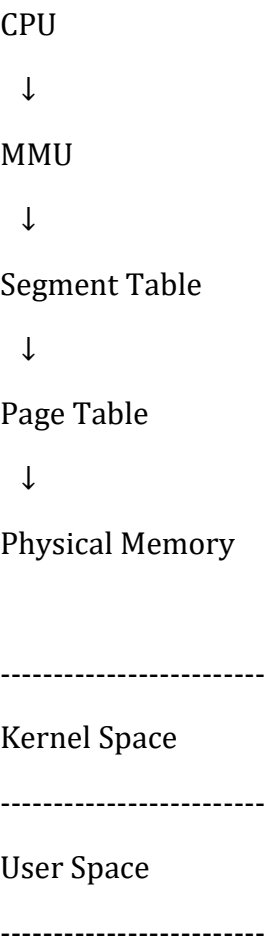
Case Analysis

A cybersecurity organization requires strong memory protection between user and kernel processes. Analyze how **segmentation, paging, and segmentation with paging** provide isolation and design a secure memory architecture.

Comparison

Technique	Protection Memory Utilization	
Segmentation	High	Moderate
Paging	Good	High
Segmentation with Paging	Very High	Very High

Secure Memory Architecture



Security Features

- Separate kernel and user address spaces.
- Access permissions (Read, Write, Execute).
- Memory protection bits.
- Page-level protection.
- Address translation through MMU.

Recommendation

Segmentation with Paging offers the highest level of security because it combines logical separation with page-level protection, preventing unauthorized memory access while maintaining high execution efficiency.

Question 18

Case Analysis

A fintech company is designing an operating system for financial servers. The system must support secure memory allocation, efficient page replacement, shared libraries, and minimal disk I/O. Develop a complete memory management framework.

Memory Management Framework

1. Dynamic Storage Management
 - Efficient heap allocation.
 - Reduce memory wastage.
2. Dynamic Linking
 - Shared libraries loaded only when required.
 - Saves RAM.
3. Paging
 - Eliminates external fragmentation.
4. Virtual Memory
 - Supports very large applications.
5. Demand Paging
 - Loads only required pages.
6. LRU Page Replacement
 - Keeps frequently used pages in memory.
7. Working Set Model
 - Prevents thrashing.
8. SSD Storage
 - Faster swapping.
 - Reduced disk access time.

Architecture

Application



Dynamic Linking



Virtual Memory



Demand Paging



LRU Page Replacement



SSD Storage

Recommendation

Integrating dynamic linking, paging, virtual memory, LRU page replacement, working sets, and SSD-based storage provides high security, efficient memory usage, fast execution, and minimal disk I/O for financial servers.

Question 19

Case Analysis

An autonomous vehicle runs **perception, navigation, and object detection** simultaneously using limited embedded memory. Analyze how **demand paging, page replacement, virtual memory, and working sets** affect real-time performance and recommend a suitable strategy.

Analysis

Technique

Effect

Demand Paging

Saves memory but may increase page faults

Virtual Memory

Supports larger applications

LRU/Clock Page Replacement Reduces unnecessary replacements

Working Set Model

Keeps active pages in RAM

Challenges

- Limited embedded memory.

- Strict real-time response.
- Very low latency required.
- High reliability.

Customized Memory Strategy

- Preload safety-critical modules.
- Use demand paging only for non-critical modules.
- Apply the Working Set Model.
- Use the Clock algorithm for fast page replacement.
- Store swap files on SSD if available.
- Reserve dedicated memory for navigation and object detection.

Recommendation

A hybrid strategy with preloaded critical modules, working-set-based allocation, Clock page replacement, and limited demand paging provides deterministic execution and minimal latency for autonomous vehicles.

Question 20

Case Analysis

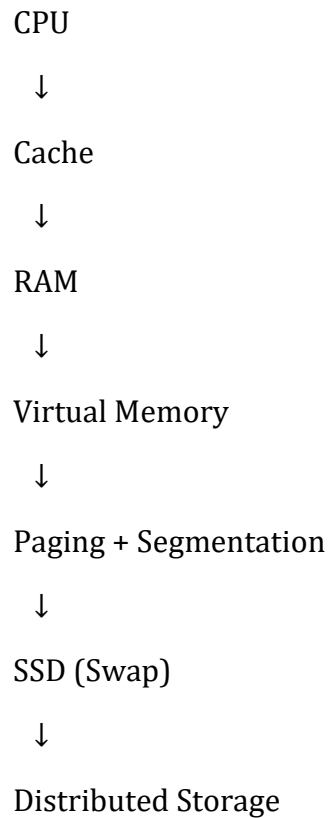
A technology company plans to build an operating system for future **AI-powered supercomputers** supporting millions of concurrent processes, shared libraries, heterogeneous memory devices, SSDs, and distributed storage. Design a complete memory management architecture and evaluate its trade-offs.

Complete Memory Management Architecture

- **Dynamic Linking** for shared libraries.
- **Dynamic Storage Management** for flexible memory allocation.
- **Paging** to eliminate external fragmentation.
- **Segmentation with Paging** for logical organization and protection.
- **Virtual Memory** to support very large AI applications.
- **Demand Paging** to load only required pages.
- **LRU Page Replacement** to reduce page faults.
- **Working Set Model** to prevent thrashing.

- **SSD-based Swapping** for faster page transfers.
- **Distributed Storage** for scalability and reliability.

Architecture Diagram



Trade-offs

Feature	Benefits	Trade-offs
Dynamic Linking	Saves memory	Small loading overhead
Paging	Eliminates external fragmentation	Page table overhead
Segmentation with Paging	Better protection	More complex address translation
Virtual Memory	Supports large applications	Possible page faults
SSD Swapping	Faster than HDD	Higher cost
Working Set Model	Prevents thrashing	Continuous monitoring required

Feature	Benefits	Trade-offs
Distributed Storage	High scalability and reliability	Network communication overhead

Final Recommendation

The operating system should integrate **dynamic linking, dynamic storage management, segmentation with paging, virtual memory, demand paging, LRU page replacement, working-set-based memory allocation, SSD-based swapping, and distributed storage**. This architecture provides excellent **performance, scalability, reliability, security, and efficient memory utilization**, making it suitable for future AI-powered supercomputers.